

DataGridle – End-to-End Experience Overview

1. High-Level Vision

DataGridle is a full-stack, LeetCode-style platform focused on interview preparation for data roles (Data Engineering, Data Science, Machine Learning, MLOps, DevOps/DataOps, Generative AI, Business Intelligence, Data Architecture, NLP/Computer Vision, and more).

It combines:

- **Domain-first learning** – users pick a data domain up-front.
- **Structured roadmaps** – week-by-week learning paths for each domain.
- **Curated study resources** – embedded YouTube video playlists per domain.
- **Hands-on practice** – SQL/Python/analytics questions with an in-browser SQL editor.
- **Role-aware access** – user / creator / admin roles with approvals and moderation.

The project is implemented as:

- **Frontend:** Next.js 14 + React + TailwindCSS.
 - **Backend:** FastAPI + SQLAlchemy 2.0 + PostgreSQL + Redis.
 - **Infrastructure:** Docker Compose for local dev, seed scripts for initial content.
-

2. User Journeys Implemented

2.1 Anonymous Visitor → Domain-First Landing Status: Completed

- Landing page (/) is a **domain picker**.
- Domains (10+) are fetched from the backend and rendered as large, clickable cards:
 - Data Engineering, Data Science, Machine Learning, Data Analytics, MLOps, DevOps/DataOps, Generative AI / LLM Engineering, Business Intelligence, Data Architecture, NLP / Computer Vision.
- Each card shows:
 - Domain name and short description.
 - Custom icon & color theme.
- Primary CTA is “**Pick Your Domain**”, scrolling to the domain grid.

Outcome: even without logging in, visitors clearly see all available paths and can explore any domain hub.

2.2 Domain Hub Experience (/domain/[slug]) Status: Completed

Each domain has a dedicated **hub page** with three tabs:

1. Roadmap

- Shows a **week-by-week timeline** for the domain.
 - Each roadmap item displays:
 - Week number.
 - Title and description.
 - Estimated study hours.
 - Roadmap is backed by a dedicated `roadmap_items` table and is ordered by `order_index` / `week_number`.
2. **Learn**
- Shows a grid of **embedded YouTube video cards**.
 - Each card displays:
 - Video title.
 - Duration (if known).
 - Click-through link to the original YouTube video.
 - Videos are stored as `learning_resources` in the backend, linked to both a category and (optionally) a roadmap item.
3. **Practice**
- Lists **practice questions** that belong to the selected domain.
 - Each row shows title, difficulty badge, type (SQL/Python/MCQ/Case Study), and a short description.
 - Clicking a question navigates to `/question/{id}` where the full question and SQL editor are available.

The domain hub fetches everything via new backend endpoints:

- GET `/api/v1/categories/{slug}/detail`
- GET `/api/v1/categories/{slug}/roadmap`
- GET `/api/v1/categories/{slug}/resources`
- GET `/api/v1/categories/{slug}/questions`

2.3 Authentication and Roles **Status: Completed**

- **Registration (`/auth/register`)**
 - Users can sign up with:
 - * Username, email, password.
 - * Role **selector**: Learner (`user`) or Creator (`creator`).
 - Learner accounts:
 - * Created with role `USER`.
 - * Marked `is_verified = True`.
 - * Get **immediate** access.
 - Creator accounts:
 - * Created with role `CREATOR`.
 - * Marked `is_verified = False`.
 - * Must be approved by an admin before they can log in and publish questions.
- **Login (`/auth/login`)**
 - Username/password based login using JWT access + refresh tokens.
 - Backend enforces:

- * Only users with `is_verified = True` may log in.
- * Unapproved creators receive a 403 with a clear message.
- **Admin**
 - A default admin is seeded on startup:
 - * Email: `admin@datagridle.com`
 - * Password: `admin123`
 - Admin accounts have full access to admin APIs and can approve creators and questions (admin panel APIs and UI stubs exist in the architecture).

2.4 Global Auth State & Navigation **Status: Completed**

- A dedicated React **Auth Context**:
 - Reads tokens from `localStorage`.
 - Calls `/auth/me` on app load to populate the current user.
 - Exposes `login`, `logout`, and `refresh` handlers.
- The entire app is wrapped in this provider at the root layout.
- The **Navbar** consumes the context and:
 - Shows **Log In / Sign Up** when unauthenticated.
 - Shows a user avatar / profile dropdown when logged in, including links to:
 - * Dashboard.
 - * Profile.
 - * Admin panel (for admins).
 - * Logout.

Outcome: login status is reflected instantly across all pages without manual prop-drilling.

2.5 Dashboard Experience (`/dashboard`) **Status: Completed (v1)**

- Shows high-level stats cards (streak, questions solved, accuracy, time practiced). Currently these are placeholders to be wired to real metrics.
- “Daily Challenge” card linking into the questions list.
- **Domain section**:
 - Fetches all categories from the backend.
 - Renders them as smaller domain cards with icons and descriptions.
 - Each card links to its corresponding domain hub (`/domain/[slug]`).
 - Progress bars are present as UI skeletons (0% for now), ready to be backed by `user_progress` data in a future iteration.

2.6 Question Detail + SQL Editor **Status: Completed (MVP)**

- Question detail page (`/question/{id}`) shows:
 - Problem statement and table schema.
 - Example data / expected output.
 - In-browser SQL editor panel.

- **SQL Editor:**
 - Uses Monaco Editor (@monaco-editor/react), but only renders on the client to avoid SSR issues.
 - Falls back to a simple `<textarea>` if Monaco cannot load.
 - Integrates with a DuckDB-WASM based execution flow (architecture in place for client-side evaluation of SQL against practice datasets).

Outcome: users can type SQL solutions directly in the browser and get feedback without needing any local environment.

3. Backend Capabilities Implemented

3.1 Data Model Extensions Status: Completed

- **RoadmapItem** model:
 - Represents a single step in a domain roadmap (week number, title, description, estimated hours, ordering).
 - Linked to a **Category** via `category_id`.
- **LearningResource** model:
 - Extended to include:
 - * `roadmap_item_id` (optional FK to **RoadmapItem**).
 - * Rich metadata for YouTube and other resources.
- **Category** model:
 - Extended with a `roadmap_items` relationship for easy loading of per-domain roadmaps.

These changes are fully registered in the SQLAlchemy metadata and used in API responses.

3.2 Seed Data Status: Completed (rich initial dataset)

- On first startup, the backend seeds:
 - 10 domains (`categories`).
 - 8–12 topics per domain.
 - 8–10 roadmap items per domain (roughly a 2-month to 10-week plan).
 - 5–7 curated YouTube resources per domain from well-known channels (freeCodeCamp, Krish Naik, TechWithTim, DataTalksClub, etc.).
 - 5–6 sample practice questions per domain across SQL, Python, MCQ, and case studies.
 - A default admin user and a curated list of company tags.

Outcome: a fresh deployment immediately feels “full” – all domains show meaningful roadmaps, resources, and starter questions without any manual content entry.

3.3 Category / Domain APIs Status: Completed

New and updated endpoints under `/api/v1/categories`:

- `GET /categories/` – list all domains.
- `GET /categories/{slug}/detail` – fetch a single domain by slug.
- `GET /categories/{slug}/roadmap` – ordered list of roadmap items.
- `GET /categories/{slug}/resources` – list of learning resources (videos).
- `GET /categories/{slug}/questions` – questions scoped to a given domain.
- Existing daily question and topics endpoints remain available for dashboard-style features.

These endpoints are what power the domain picker, domain hub, and dashboard domain lists on the frontend.

4. Infrastructure & Local Developer Experience

Status: Completed

- **Docker Compose:**
 - Spins up:
 - * `frontend` (Next.js dev server).
 - * `backend` (FastAPI + Uvicorn).
 - * `postgres` (primary DB).
 - * `redis` (cache/session placeholder).
 - Backend automatically runs migrations (via SQLAlchemy metadata create) and seed logic on startup.
- **Environment Configuration:**
 - Sample variables documented in `.env.example` for both backend and frontend.
 - Local development supports both:
 - * Docker-based workflow (`docker-compose up --build`).
 - * Manual workflow (Python virtualenv + `uvicorn`, `npm run dev`).

Outcome: any developer can clone the repo and spin up a fully seeded, domain-first DataGridle instance with a single Docker command.

5. Summary of What’s Ready to Demo

From a “show this to someone” perspective, the following end-to-end flows are demo-ready:

- Landing on / and picking a data domain from a visually rich domain grid.
- Exploring a domain hub:
 - Viewing the multi-week roadmap.
 - Playing curated YouTube lessons.
 - Browsing domain-specific practice questions.

- Signing up as a **Learner** or **Creator** and seeing role-sensitive behavior:
 - Learners get immediate access.
 - Creators must be approved by an admin before login.
- Logging in and seeing the Navbar and Dashboard reflect the authenticated state.
- Opening a question detail page with the integrated SQL editor and understanding how in-browser SQL practice will work.
- Explaining how admins and creators fit into the ecosystem using the seeded admin account and the existing role logic.

The system is therefore suitable as a **functional prototype** and **portfolio-grade demo** for a domain-first, end-to-end data interview preparation platform.